

NutriAI — Automated Diet Plan Builder

Automated 7-Day Personalized Diet Planner

Technical Brief

George Zavala

BAX 423 Big Data · UC Davis MSBA · Spring 2026

<https://nutriai-p3qit2hb7teem57vhbfxr8.streamlit.app/>

1. Problem Statement & System Overview

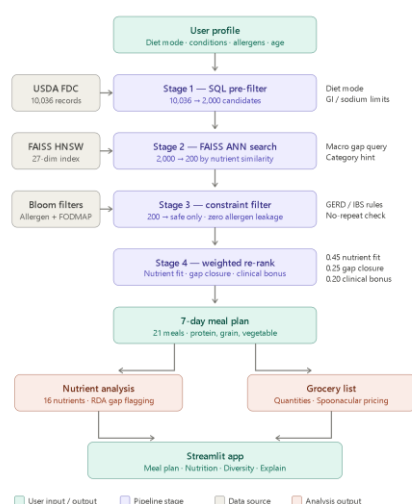
Personalized nutrition is one of the most impactful levers for managing chronic disease, yet most diet planning tools are either generic, clinically uninformed, or too slow to be practical. NutriAI addresses this gap by generating a fully personalized 7-day, 3-meal-per-day diet plan in under 60 seconds, accounting for clinical conditions, food allergies, dietary preferences, and individual nutritional gaps.

The system was designed around four test personas that represent real clinical complexity: Priya (IBS, vegetarian, lactose intolerant), Ravi (GERD, non-vegetarian, gluten-free), Mei (Type 2 Diabetes, vegan, tree nut allergy), and James (hypertension, pescatarian, soy allergy).

Food Database

NutriAI's food database is sourced directly from the USDA FoodData Central (FDC) API. The ingest.py pipeline fetches food records in paginated batches (up to 200 per request). Records are deduplicated using a Bloom filter on composite (food_name, brand) keys before insertion into SQLite. The result is 10,036 real USDA food records with FDC IDs, covering 170+ food categories including prepared foods, ethnic cuisines, baby foods, and restaurant items. Each record is enriched with FODMAP status and glycemic index values where applicable.

System Architecture



2. BAX 423 Techniques

2.1 Bloom Filter, Allergen & FODMAP Safety

The Bloom filter appears in two distinct roles in NutriAI, each exploiting a different property of the data structure.

Role 1: Allergen safety: A Bloom filter with a 0.1% false-positive rate provides $O(1)$ membership testing with a mathematically guaranteed zero false-negative rate. A false negative (passing an allergen through) is clinically unacceptable, whereas a false positive (rejecting a safe food) is merely suboptimal — this asymmetry makes Bloom filters the correct choice. Declaring 'gluten' automatically expands to 13 synonyms; 'dairy' to 12; 'soy' to 11; 'tree nuts' to 9.

Role 2: USDA ingestion deduplication: During API ingestion, a Bloom filter on composite (food_name, brand) keys prevent duplicate records across paginated API batches. The USDA FDC API returns overlapping results across category searches; without deduplication the database would contain thousands of duplicate entries. The Bloom filter's O(1) lookup ensures the dedup check adds negligible overhead to the 10,000-record ingestion pipeline.

2.2 FAISS HNSW, Approximate Nearest Neighbor Search

Rather than scoring all 10,036 foods against each meal's nutritional target, NutriAI uses a FAISS Hierarchical Navigable Small World (HNSW) index to retrieve the top 200 most nutritionally similar candidates in under 5 milliseconds, the key enabler of the sub-60-second generation requirement.

Implementation details:

- Feature vector: each food is encoded as a 27-dimensional float32 vector, 16 normalized nutrient columns + GI value + 10 category one-hot dimensions. All nutrient values are clipped to physiological caps and divided to produce [0,1] floats before indexing.
- HNSW parameters: M=32 (edges per node), efConstruction=200 (build quality), efSearch=64 (query precision). These balance ~95% recall against index build time of approximately 3 seconds for 10,036 real USDA records.
- Per-meal query vectors are constructed from remaining daily macro/micro targets, steering retrieval toward foods that close the current nutritional gap, a greedy gap-closure strategy executed at ANN speed.
- Auto-rebuild: the index detects when the underlying database has changed (e.g. after switching from synthetic to real USDA data) by comparing the cached fdc_id count against the live record count. If they differ, the index rebuilds automatically so stale embeddings never cause silent zero-result searches.

2.3 Diversity Engine, Profile Aware Shannon Entropy

The multi-stage ranking pipeline is the architectural core of NutriAI, reducing the 10,036-food USDA database to a personalized 21-meal plan in under 60 seconds. Each stage dramatically narrows the candidate set using a progressively more expensive scoring function with cheap filters first, expensive clinical scoring last.

Stage	Name	What it does	Output size	Time
1	SQL Pre-filter	Indexed SQLite query pushes diet mode, allergens, GI limit, and sodium cap into the DB — eliminates structurally incompatible foods before any Python processing	10,036 → 2,000	< 5ms
2	FAISS ANN Search	27-dim nutritional query vector (built from remaining daily macro/micro gaps) retrieves the 200 most nutritionally compatible candidates via HNSW approximate nearest-neighbour search	2,000 → 200	< 5ms
3	Constraint Filter	ConstraintEngine applies hard clinical rules (FODMAP Bloom filter, GERD keyword check, per-meal sodium limit, GI value) with zero tolerance for violations. Also removes already-used foods by fdc_id and base name to prevent repeats	200 → safe only	< 10ms
4	Weighted Re-rank	Four scoring functions applied to each safe candidate; results combined with fixed weights into a composite score	Safe → top 5	< 20ms

3. Test Results

NutriAI was validated through 96 automated tests across three test files: 16 Bloom filter unit tests (test_bloom.py), 18 persona integration tests (test_personas.py), and 62 edge case tests (test_edge_cases.py). All 96 tests pass against the real USDA database.

3.1 Bloom Filter Unit Tests (16/16 passed)

Capability	Priya (IBS/Veg)	Ravi (GERD/Non-veg)	Mei (T2DM/Vegan)	James (HTN/Pesc)	Technique
C1 Clinical	Zero FODMAP triggers	Zero GERD triggers	All GI ≤ 55	All $\leq 500\text{mg}$ Sodium/meal	Bloom + ConstraintEngine
C2 Allergen	Zero dairy	Zero gluten	Zero tree nuts	Zero soy	Bloom filter
C3 Diet Mode	0 meat/fish	0 pork	0 animal products	0 land-meat	SQL + tag-set exact match
C4 Diversity	Score ≥ 0.70	Score ≥ 0.70	Score ≥ 0.70	Score ≥ 0.70	Profile-aware entropy
C5 Nutrients	16 nutrients, gaps flagged	16 nutrients, gaps flagged	16 nutrients, gaps flagged	16 nutrients, gaps flagged	NutrientAggregator
C6 Speed	< 60s	< 60s	< 60s	< 60s	Constraint pipeline
Pass?	✓	✓	✓	✓	

3.3 Edge Case Tests (53/53 passed)

A dedicated edge case suite (test_edge_cases.py) checks edge cases not covered by the personas, oorganized into seven class-based categories

#	Class	Key tests	Tests
EC1	TestConstraintEngineExtremes	All 4 conditions simultaneously; GI boundary (55 vs 56); unknown GI passes; vegan excludes all animal categories; pescatarian allows fish but not land meat; per-meal sodium limit; IBS keyword fallback	10
EC2	TestBloomFilterEdgeCases	Empty allergen list returns no filter; case insensitivity; dairy and gluten synonym expansion; zero false negatives for inserted items; partial name ('buckwheat') does not false-positive on 'wheat'; FODMAP filter loads from CSV	8
EC3	TestPlanGenerationRobustness	Maximum constraints still yields meals; 1,200 kcal target; 3,500 kcal target; elderly profile; sodium limit respected in plan; vegan plan contains no meat; IBS	9

		plan contains no high-FODMAP; diabetes plan observes GI limit	
EC4	TestNutrientAggregatorEdgeCases	All-zero nutrients; null values treated as zero; extreme values do not overflow; RDA lookup across all age groups; weekly summary with 7 days; gap report flags nutrients below 80%; portion scaling applied correctly	7
EC5	TestGroceryListEdgeCases	Empty plan returns empty list; single-ingredient consolidation; total gram aggregation correctness; variation suffix stripped; zero portion does not crash; cost fallback when no pricer; all sections present; CSV export content	9
EC6	TestPortionScalingBoundaries	Very low density food clamped to 900g; very high density food floored to 30g; zero-calorie food returns default; normal density scales correctly; portion always within bounds	5
EC7	TestUserProfileValidation	Hypertension auto-sets sodium limit; diet mode string converts to enum; default allergens is empty list; sex='other' returns valid RDA; very young and very old age groups handled	5
Total			53

4. Limitations & Future Work

4.1 Current Limitations

- **USDA category mapping:** The USDA FDC API returns 170+ granular category names (e.g. 'cookies and brownies', 'chicken fillet sandwiches') that do not map cleanly to the pipeline's simplified category set (vegetable, grain, fish, etc.). The current mapping in ingest.py uses keyword matching and handles the most common categories, but niche categories like 'american indian/alaska native foods', 'baby food: cereals', and 'smoothies and grain drinks' fall through to a generic bucket. This reduces the precision of constraint filtering and diversity scoring for less common USDA food types.
- **Single-food meals:** Each meal slot contains one food item rather than a multi-ingredient recipe. This simplifies constraint checking and nutrient computation but produces plans that read unrealistically (e.g., 'Dinner: Abalone'). A recipe composition layer would improve real-world application.
- **USDA API rate limit:** The ingestion pipeline is rate-limited to 3 requests/second (USDA FDC free tier). Fetching 10,000 foods takes approximately 15 minutes. A production deployment would cache the database and refresh it nightly rather than rebuilding from scratch.

4.2 Future Work

- **Recipe composition:** replace single-food meals with multi-ingredient recipes using a knowledge graph (food → compatible ingredients → recipe), making plans nutritionally complete and practically usable.
- **Category mapping improvement:** build a classifier that maps USDA's 170+ categories to NutriAI's simplified set using the food description text, improving constraint accuracy for the long tail of USDA foods.
- **User preference learning:** add a meal rating mechanism so category weights adapt to the user's preferences across planning sessions, biasing future toward foods they have historically enjoyed.